Claude Code Mods and the 'You Should Know' Side-Agent: Research & Verification Guide

Researched 2026-10-12.

Overview: Claude Code Mods & Side Agents

A social media reel by @j.t.rivard on Instagram (https://www.instagram.com/reel/DeRQMvaBZY4/) claims that Anthropic shipped a powerful, disabled-by-default side-agent mod called "You should know" in Claude Code. The creator asserts that this background agent monitors terminal transcripts, warns developers of missed edge cases via a prompt-line banner, and can be activated via a direct-message link.

Evaluating these claims against Anthropic's official Claude Code documentation (https://code.claude.com/docs/en/plugins/mods/overview) reveals a divide between actual architectural features (the Claude Mods system) and unverified engagement marketing.

---

1. What the Reel Claims

- **Silent Side-Agent Mod:** A built-in plugin named "You should know" observes your active session transcripts and watches for potential oversights while Claude executes longer tasks.
- **Interactive Prompt Notice:** A "Heads up" notification appears above the terminal prompt; typing or pressing `1` ("Learn more") displays an explanation.
- **Strict Operating Constraints:** The feature reportedly requires a direct, first-party Claude Code login and enabled usage reporting/telemetry.
- **Gated Setup Link:** The creator prompts viewers to comment "CLAUDE CODE" to receive a direct message containing an automated setup link.

---

2. What Official Documentation Confirms

According to Anthropic's Claude Code Mods Overview (https://code.claude.com/docs/en/plugins/mods/overview):

- **Internal Mods Architecture:** Claude Code includes an internal runtime called "mods" (available in terminal from v2.1.287 and Desktop from v2.1.286). Unlike settings hooks or external MCP servers, mods run inside Claude Code and can directly manipulate UI elements, intercept tool calls, and share state across lifecycle events.
- **UI Rendering Capabilities:** Mods have explicit access to render custom interface elements, including "a band above the prompt" and side panels, corroborating the UI pattern described in the reel.
- **Built-in Mods Management:** Claude Code includes built-in mods (such as `diff`, `telemetry`, `sec-default`, and `agents-md`). Users can inspect built-in mods via the interactive `/plugin` interface under the **Installed** tab.
- **Usage and Cost Model:** Official documentation explicitly warns that mods run with your user permissions and can "spend your usage: call a model on your plan or API key."

- **No DM/Gated Setup Required:** Official plugins and mods are managed natively using the built-in `/plugin` command or CLI flags, confirming the creator's DM-gated setup is promotional engagement harvesting.

---

3. What Remains Uncertain & Unverified

- **`cc-plugin-you-should-know@builtin` Availability:** The specific identifier and enablement command (`/plugin enable cc-plugin-you-should-know@builtin`) appear in user dossiers and changelogs cited by the reel, but the fetched official mods documentation lists only sample and core mods (`diff`, `agents-md`, `sec-default`, `telemetry`). Its presence across all account tiers remains unverified.
- **Keybinding `1` for Plain Explanation:** While the mod UI supports interactive text fields and buttons, the single-key prompt handler (`1` to "Learn more") is not independently confirmed in the public docs.
- **Telemetry Coupling:** Official docs indicate `telemetry` is a built-in mod providing shared helper methods, but the specific technical gate tying watcher capabilities to active telemetry reporting is unverified in primary sources.

---

4. Prerequisites, Limits & Common Mistakes

Prerequisites

- **Claude Code Version:** v2.1.287 or higher for terminal CLI, or v2.1.286 or higher for Claude Desktop.
- **Supported Terminal/Desktop:** Visual drawing bands only render in terminal shells and the Desktop app's Code tab. They do not render in headless `claude -p` sessions or standard VS Code chat panels.

Realistic Limits

- **Token & Billing Overhead:** Background reviewer agents make extra model calls against your API allowance or subscription plan. Running an active watcher across high-frequency tasks can substantially increase token consumption.
- **Non-Deterministic Oversight:** An LLM watcher reviewing another LLM remains prone to both false positives (nagging on intentional patterns) and false negatives.
- **Security Boundary:** A mod runs with your local user permissions. It can inspect environment variables, read API keys, rewrite tool calls, and execute network requests.

Common Mistakes

- **Falling for DM Link Traps:** Giving permissions or scripts from third parties when native mods are inspected entirely via `/plugin`.
- **Running Headless or Subshell Environments:** Expecting interactive prompt bands to draw in unsupported environments like `claude -p`.
- **Confusing Deterministic Tests with LLM Watchers:** Relying on a probabilistic side agent instead of deterministic `PreToolUse` hooks, linters, or CI test suites.

---

5. Verified Commands vs. Unverified Reel Commands

Verified Commands (from Official Docs)

- Check installed CLI version: `claude --version`
- Open plugin and mod management UI: `/plugin`
- Check desktop session status and version: `/status`
- Launch isolated session without mods: `claude --safe-mode`

Unverified Reel / Dossier Command

- Enable watcher plugin directly: `/plugin enable cc-plugin-you-should-know@builtin` *(Unverified in current core documentation; inspect `/plugin` interactively instead).*

Original sources

- https://www.instagram.com/reel/DeRQMvaBZY4/